

The School on Quantum Optimization Algorithms (SQOA) 2026

Environment Setup Tutorial for Python, Anaconda, Qiskit, and PyCharm

M.A. Quoc Chuong Nguyen & PhD. Vu Tuan Hai



2026 SCHOOL ON QUANTUM OPTIMIZATION ALGORITHMS

22 – 25/4 University of Phan Thiet, Phu Thuy, Lam Dong

The School on Quantum Optimization Algorithms (SQOA) 2026 is a specialized program for undergraduate and graduate students interested in quantum optimization algorithms. The course provides a theoretical foundation, the latest results, and the implementation of quantum optimization algorithms through hands-on projects.

Scientific Advisors
Prof. Nguyen Minh Tho, VinUni
Prof. Le Van Hoang, HCMUE

Lecturer
Assist.Prof. Le Bin Ho, Tohoku Univ
Dr. Nguyen Van Duy, Phenikaa Univ
Dr. Huynh Minh Trung, US-VNUHCM
Dr. Vu Tuan Hai, UIT-VNUHCM
MA. Nguyen Quoc Chuong, DTU

Organizing Committee
Dr. Tran Nguyen Lan, US-VNUHCM
Dr. Nguyen Minh Tam, UPT
Msc. Truong Duc Toan, VLU

Date 1:
Foundation for quantum computing

Date 2:
Basic quantum computing

Date 3:
Project: Q-optimization



Email: quantum.vnuhcm@gmail.com

Registration deadline:
31/03/2026

1 Purpose of This Tutorial

This tutorial explains how to set up the software environment for the school in quantum optimization. The programming language used in this course is **Python**, and the main software tools are: Python, Anaconda, PyCharm, and the Qiskit family. The objective is not only to install the required software, but also to verify that the installation works correctly by running a simple quantum circuit.

By the end of this tutorial, you should be able to:

1. Install Anaconda,
2. Create and activate a dedicated Python environment,

3. Install Qiskit and its supporting packages,
4. Configure PyCharm to use the correct Python interpreter,
5. Run your first Qiskit program successfully.

Recommended System Requirements

- Operating system: Windows 10/11, macOS, or Linux,
- At least 8 GB RAM recommended,

2 Why We Use This Software Stack

Before starting the installation, it is useful to understand the role of each tool.

2.1 Python

Python is the programming language used throughout the course. It is widely used in scientific computing, optimization, and quantum computing.

2.2 Anaconda

Anaconda helps manage Python installations and virtual environments. This is important because different projects may require different package versions. Using a dedicated environment prevents conflicts with other Python projects on your computer.

2.3 Qiskit

Qiskit is an open-source software development kit for quantum computing. In this summer school, it will be used to build circuits, simulate quantum systems, and study optimization-related quantum algorithms.

2.4 PyCharm

PyCharm is a powerful IDE for Python. It makes it easier to write, organize, run, and debug code, especially in structured course projects.

3 Step 1: Install Anaconda

Step 1.1: Download Anaconda. Go to the official Anaconda download page:

<https://www.anaconda.com/download>

Download the version appropriate for your operating system: Windows, MacOS, Linux [Skip Registration].

Step 1.2: Run the Installer. After downloading the installer, run it and follow the instructions on screen.

For Windows

- Choose the standard installation unless you have a special reason to change it.
- If asked whether to install for “Just Me” or “All Users”, choose according to your permissions. “Just Me” is usually sufficient.
- You may see an option related to adding Anaconda to PATH. In many cases, Anaconda recommends not adding it automatically. This is acceptable.

For macOS and Linux

- Follow the command-line or graphical installation instructions.
- If the installer asks to initialize your shell, choose yes.

Step 1.3: Verify the Installation. Open a terminal (On Windows: Anaconda Prompt, Command Prompt, or PowerShell, On macOS/Linux: Terminal). Run:

```
conda --version
python --version
```

If the command `conda --version` fails, try opening **Anaconda Prompt** instead of a regular terminal. You should see output similar to

```
conda 24.x.x
Python 3.x.x
```

4 Step 2: Create a Dedicated Python Environment

For this course, Python 3.10 is a safe and stable choice for compatibility with the Qiskit ecosystem and supporting scientific packages. It is strongly recommended that you do *not* install course packages into the base environment. Instead, create a separate environment for this summer school.

Step 2.1: Create the Environment. In the terminal, run:

```
conda create -n quantum_env python=3.10
```

When prompted with something like: Proceed ([y]/n)?, type: y.

Step 2.2: Activate the Environment. After the environment is created, activate it using:

```
conda activate quantum_env
```

Once activated, your terminal should show the environment name at the beginning of the line, for example: `(quantum_env) C:\Users\YourName>`

Step 2.3: Verify That the Environment Is Active. Run: `python --version` and then: `which python`. On Windows, use `where python`. The returned path should point to the `quantum_env` environment rather than the system Python installation.

5 Step 3: Install Qiskit and Supporting Packages

Step 3.1: Install Core Packages Run the following commands:

```
pip install "qiskit>=1.0,<2.0" qiskit-aer qiskit-algorithms==0.4.0 matplotlib
```

- `qiskit`: core quantum programming framework,
- `qiskit-aer`: simulator backend for running circuits locally,
- `qiskit-algorithms`: for advanced quantum algorithms,
- `matplotlib`: plotting library used for visualizing results.

Step 3.2: Verify the Installation by run:

```
python -c "import qiskit; print(qiskit.__version__)"
python -c "from qiskit_aer import AerSimulator"
```

If both commands run without error, the installation is successful.

6 Step 4: Install PyCharm

Go to: <https://www.jetbrains.com/pycharm/>. Download **PyCharm Community Edition** unless you already have the Professional Edition. Run the installer and follow the instructions. Default settings are usually sufficient.

After installation: launch PyCharm, then create a new project or open an existing one for the course.

7 Step 5: Configure PyCharm to Use the Correct Python Environment

This step is essential. Many setup errors happen because PyCharm is using the wrong Python interpreter.

Step 5.1: Open Interpreter Settings In PyCharm, navigate to:

Settings → Project → Python Interpreter

On macOS, this may appear under:

PyCharm → Settings → Project → Python Interpreter

Step 5.2: Select the Conda Environment Choose:

- **Conda Environment**,
- then select **Existing environment**,
- browse to the Python executable inside `quantum_env`.

Typical locations are:

Windows

```
C:\Users\YourName\anaconda3\envs\quantum_env\python.exe
```

macOS/Linux

```
/Users/YourName/anaconda3/envs/quantum_env/bin/python
```

or

```
/home/YourName/anaconda3/envs/quantum_env/bin/python
```

Step 5.3: Apply the Configuration After selecting the interpreter:

- click OK or Apply,
- wait a few seconds for PyCharm to index the environment.

Step 5.4: Verify the Interpreter in PyCharm Open the terminal inside PyCharm and run:

```
python --version
```

Then run:

```
python -c "import qiskit; print(qiskit.__version__)"
```

If these work, then PyCharm is correctly configured.

8 Step 6: Create and Run the First Quantum Program

We now verify the full setup by running a simple one-qubit quantum circuit.

Step 6.1: Create a New Python File In PyCharm:

- right-click your project folder,
- choose New → Python File,
- name the file `first_qiskit_program.py`.

Step 6.2: Paste the Following Code

```
from qiskit import QuantumCircuit
from qiskit_aer import AerSimulator
from qiskit.visualization import plot_histogram
import matplotlib.pyplot as plt

qc = QuantumCircuit(1, 1)
qc.h(0)
qc.measure(0, 0)
simulator = AerSimulator()
result = simulator.run(qc, shots=1000).result()
counts = result.get_counts()

print("Measurement counts:", counts)
plot_histogram(counts)
plt.show()
```

```
import numpy as np
from qiskit.circuit.library import EfficientSU2
from qiskit.quantum_info import SparsePauliOp
from qiskit_algorithms import VQE
from qiskit_algorithms.optimizers import SLSQP
from qiskit.primitives import StatevectorEstimator

hamiltonian = SparsePauliOp.from_list([("ZZ", 1.0)])

ansatz = EfficientSU2(num_qubits=2, reps=1, entanglement='linear')

estimator = StatevectorEstimator()
optimizer = SLSQP(maxiter=100)

vqe = VQE(estimator, ansatz, optimizer)
result = vqe.compute_minimum_eigenvalue(operator=hamiltonian)

print(f"VQE Eigenvalue: {result.eigenvalue.real:.5f}")
print(f"Optimal Parameters: {result.optimal_point}")
```

Step 6.3: Interpret the Output Click the run button in PyCharm, or run the file from the terminal with: `python first_qiskit_program.py`. You should see two kinds of output.

```
Measurement counts: {'0': 493, '1': 507}
---
VQE Eigenvalue: -1.00000
Optimal Parameters: [ 8.77041599e-01 -1.22820166e+00  2.14629190e-04  1.32561676e-04
 8.47396447e-01 -1.79528574e+00 -2.08405096e+00  2.70565267e+00]
```

9 Step 7: Verification Checklist

At this point, confirm all of the following:

- ☐ Anaconda is installed correctly,
- ☐ The environment `quantum_env` was created,
- ☐ The environment can be activated from the terminal,
- ☐ Qiskit imports without errors,
- ☐ `qiskit_aer` imports without errors,
- ☐ PyCharm uses the `quantum_env` interpreter,
- ☐ The example quantum program runs successfully,
- ☐ The histogram appears correctly.

10 Step 8: Common Problems and Solutions

Problem 1: conda Command Not Found. Anaconda is not initialized in your terminal.

Possible solutions:

- On Windows, use **Anaconda Prompt**,
- restart the terminal after installation,
- reinstall Anaconda if necessary.

Problem 2: ModuleNotFoundError: No module named 'qiskit'. Qiskit was installed in a different environment.

Solution:

```
conda activate quantum_env  
pip install qiskit qiskit-aer matplotlib
```

Then verify again:

```
python -c "import qiskit"
```

Problem 3: PyCharm Runs the Wrong Python. The project interpreter is not set to `quantum_env`.

Solution: Go back to:

Settings → Project → Python Interpreter

and explicitly choose the interpreter inside the course environment.

Problem 4: No Plot Window Appears. The plotting backend may not be responding properly.

Solution: Make sure your code contains:

```
plt.show()
```

You may also try running the script directly from the terminal.

Problem 5: Installation Is Slow or Interrupted. Internet connection problems or package server delays.

Solution: Retry the installation commands after a few minutes. In most cases, this is temporary.

Problem 6: Apple Silicon Compatibility Issues. Some packages behave differently on M-series chips.

Solution: Use Python 3.10 and install packages inside a clean conda environment as described above.

Optional Section: Jupyter Notebook Setup

Although the main programming workflow of this course uses PyCharm, some students may also want to use notebooks for experimentation.

10.1 Install Jupyter

If not already installed:

```
pip install jupyter
```

10.2 Launch Jupyter

Run:

```
jupyter notebook
```

This will open a notebook interface in your browser.

10.3 Verify Qiskit in a Notebook

Create a new notebook and run:

```
import qiskit
print(qiskit.__version__)
```

Optional Section: IBM Quantum Runtime

If the course later includes access to IBM Quantum cloud resources, an additional package may be needed. This is not necessary for the local simulator-based setup in the present tutorial. Install it using:

```
pip install qiskit-ibm-runtime
```

Summary of Essential Commands

For convenience, the main commands in this tutorial are collected below.

```
conda create -n quantum_env python=3.10
conda activate quantum_env

pip install "qiskit>=1.0,<2.0" qiskit-aer qiskit-algorithms==0.4.0 matplotlib

python -c "import qiskit; print(qiskit.__version__)"
python -c "from qiskit_aer import AerSimulator"
```

Final Remarks

A successful setup is the first step toward productive work in quantum optimization. If your installation is correct, you now have a working environment for writing, running, and testing Qiskit programs. In the school, this environment will be used for quantum circuits, simulation experiments, and optimization-oriented quantum algorithms.

References

- Python: <https://www.python.org/doc/>
- Anaconda: <https://www.anaconda.com/docs/main>
- Qiskit: <https://quantum.cloud.ibm.com/docs/en/guides/quick-start>
- PyCharm: <https://www.jetbrains.com/help/pycharm/getting-started.html>
- Jupyter Notebook: <https://docs.jupyter.org/en/latest/start/index.html>